

Hands-On-Learning: Prompt Optimization Challenge – Make It Right and Tight

Learning Objectives

By the end of this activity, learners will be able to:

- Diagnose vague or ambiguous prompts by analyzing LLM outputs for reasoning gaps and hallucinations.
- Apply role prompting, reasoning guidance, and structured output formats (e.g., JSON) to improve LLM reliability.
- Iteratively refine prompts and compare versions using concrete criteria (factuality, structure, tone).
- Document prompt evolution and articulate lessons learned about LLM reasoning control.



Description

In this hands-on practice, you will act as a prompt engineer responsible for turning a messy, unreliable LLM prompt into a precise, production-ready specification. You'll run a baseline prompt, identify where the model “fails silently,” and iteratively tighten roles, structure, and reasoning until the assistant's answers become accurate, consistent, and machine-readable.

Structure of the Activity

Title: Hands-on Practice: Prompt Optimization Challenge – Make It Right and Tight

Dataset information / Resources Required:

→ Starter prompt (vague knowledge-assistant prompt) – provided by the instructor or created by you based on a simple FAQ domain (e.g., refund policy, onboarding FAQs).

→ Example knowledge snippet or policy block (a short text you want the model to rely on).

Tools Used:

- OpenAI API Playground or ChatGPT API
- Any text editor for saving prompt versions (e.g., VS Code, Notepad, markdown editor)

Setup Requirements:

- OpenAI account with access to the Playground or API.
- Stable internet connection.
- Ability to save files locally (e.g., prompt_v1.txt, prompt_v2.txt, etc.).

Scenario

You've joined an AI product team responsible for an LLM-powered knowledge assistant that answers questions about a company's policies and product features.

On paper, the assistant “works”: it responds in fluent language and usually sounds confident. In practice, it:

- Invents details sometimes that are not in the policies.
- Gives partial answers that skip critical steps.

- Returns responses in inconsistent formats, making it hard for the UI to render them cleanly.

Stakeholders don't want a new model-they want a better **prompt**. You've been asked to **analyze and optimize the existing prompt** so that responses are accurate, structured, and reproducible using the techniques from this module.

Objective

Produce an optimized prompt specification and comparison table showing how each iteration improved reasoning, factuality, and output structure.



Instructions for Participants

Getting Started

- 1 Open the **OpenAI API Playground** or your preferred ChatGPT API client.
- 2 Obtain or write a **starter prompt** for a knowledge assistant (keep it intentionally vague at first).
- 3 Prepare a short **reference text** (e.g., a refund policy paragraph or mini FAQ) that the assistant should rely on.
- 4 Create a working folder to save prompt versions and your comparison table (e.g., `prompt_optimization_lab/`).

Activity 1: Run and Diagnose the Baseline Prompt

Objective

Identify weaknesses in the existing prompt by observing how the model behaves with real questions.



Input/Data:

→ Starter Prompt: a simple instruction like
“You are a helpful assistant. Answer questions about our refund policy.”

→ Reference Text: a short refund policy or FAQ snippet pasted into the prompt or given as context.

→ 2–3 realistic user questions (e.g., “Can I get a refund after 35 days?”).

Steps to Complete

Step 1: Run the Baseline Prompt

- Paste your starter prompt and reference text into the Playground.
- Ask 2–3 realistic questions that a user might ask.
- Capture the model outputs (copy them into a notes file).

Step 2: Diagnose Silent Failures

For each output, look for:

→ **Factual drift:** Did the model invent any policy details not in your reference text?

→ **Reasoning gaps:** Did it skip important conditions or steps?

→ **Format inconsistency:** Are answers sometimes long paragraphs, sometimes bullet points, sometimes something else?

→ **Tone issues:** Does the assistant sound overly confident when it should be cautious?

Step 3: Write a Short Diagnostic Note

In your notes, describe 3–5 concrete issues with the baseline prompt (e.g., “doesn’t say ‘do not guess’, no clear output structure, role is too generic”).

Expected Outcome:

One “baseline” prompt version (prompt_v1) and a short diagnostic note summarizing what’s wrong with it.

Tips:

Don't overcomplicate the baseline; it's okay if it's "bad." That makes improvements easier to see.

Focus on specific behavior, not general feelings ("hallucinates refund eligibility beyond 30 days" rather than "feels off").

Activity 2: Refine, Compare, and Finalize the Prompt

Objective

Iteratively improve the prompt using structured techniques, then compare versions to select a final optimized prompt.



Input/Data:

- Baseline prompt and diagnostic notes from Activity 1.
- Same reference text and user questions for consistency.

Steps to Complete

Step 1: Design Prompt Version 2 (Structure & Role)

- Add a clear role (e.g., “You are a policy assistant for ACME Inc. You must answer only using the policy text below.”).
- Add constraints like “If the answer is not explicitly stated in the policy text, respond with ‘Insufficient information in policy’ and do not guess.”

Define a simple structure such as:

Respond in JSON with the following fields:

```
{  
  "answer": "...",  
  "policy_reference": "...",  
  "confidence": "high|medium|low"  
}
```

Save as prompt_v2.

Step 2: Test and Observe Prompt Version 2

- Re-run the same user questions from Activity 1 using prompt_v2.
- Note improvements and remaining issues (e.g., “format now consistent, but sometimes confidence is always ‘high’ even when uncertain”).

Step 3: Design Prompt Version 3 (Reasoning & Self-Check)

- Add reasoning guidance, such as:
 - “First, think through whether the answer is explicitly stated in the policy. Then, respond only with the final JSON object.”



Optionally add a self-critique pass, e.g.:

“Before writing the final JSON, check whether your answer strictly follows the policy and adjust if necessary.”

Tighten wording to discourage guessing even further.

Save as prompt_v3.

Step 4: Test and Compare All Versions

→ For each version (v1, v2, v3), record:

→ Factual correctness (relative to the policy text)

→ Output consistency (does it follow the JSON structure?)

→ Clarity and tone (e.g., honest about uncertainty)

→ Build a small **comparison table** (in Markdown, CSV, or a document) with columns like:

Version	Changes Made	Factuality	Structure	Notes on Behavior
v1	Baseline, no structure	Low	Low	Hallucinates; inconsistent
v2	Role + JSON + “do not guess”	Medium	High	Structure OK; tone too strong
v3	Reasoning + self-check instructions	High	High	More honest about uncertainty

Choose your **final optimized prompt** (likely v3, or v2 if time-limited) and save it as your production-ready version.

Expected Outcome:

- At least 3 prompt versions (prompt_v1, prompt_v2, prompt_v3).
- A comparison table summarizing what improved and why.
- A clearly identified “final” optimized prompt.

Tips:

- Keep changes between versions small and deliberate so it's easy to see their impact.
- Use the **same questions** each time; that's the only way to compare fairly.

Compile Your Findings

1 Optimized Prompt File

→ **Deliverable:** Final refined prompt achieving accurate, structured outputs.

→ **Include:**

→ Full text of your final prompt (role, constraints, format, reasoning instructions).

→ Any notes on model parameters if relevant (e.g., temperature, max tokens).

2 Prompt Comparison Table

→ **Deliverable:** A concise table comparing 3–4 prompt attempts.

→ **Include:**

→ Version identifiers (v1, v2, v3, etc.).

→ Summary of changes in each version.

→ Observed differences in factuality, structure, and tone.

→ A brief note on why the final version is preferred.

3

Reflection on Reasoning Control

Deliverable: Short written reflection.

Include:

What you learned about how small wording changes affect model behavior.

Which structured prompting techniques (role, JSON, reasoning, self-critique) had the biggest impact.

How you would approach prompt optimization differently next time in a real project.

Summary

This activity provides hands-on experience with **prompt optimization and reasoning control**. By diagnosing an unreliable baseline prompt and iteratively refining it, you've practiced how to:

- Detect silent failures caused by ambiguity and missing structure.
- Apply role prompting, constrained reasoning, and JSON formats to stabilize LLM outputs.
- Compare prompt versions systematically instead of relying on intuition alone.

Key takeaways

- LLMs obey ambiguity-if you don't define structure and constraints, they will invent them.
- Treat prompts like code: iterate, compare, and document changes.
- Small, targeted prompt edits can dramatically improve factuality, consistency, and downstream usability.

Resources

Primary Materials:

Starter prompt and policy/FAQ text (provided in the course repository or by instructor).

Reference Documentation:

OpenAI Prompt Engineering Guide (for delimiters, roles, and reasoning patterns).

Course slides for Module 1 on ambiguity, structured prompting, and optimization.

Additional Resources (Optional):

↳ Your own real-world policy or FAQ documents to try after the lab.

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.